

Lab Report: Electromagnetic Induction

Chhaya Ramnani (UID: u8336188)

Lab partners: Tsz Wing Candice Ng, Chinatsu Komada

2026-09-03

Imports

```
from dataclasses import dataclass
import matplotlib.pyplot as plt
import numpy as np
from scipy import odr, constants, stats
from uncertainties import ufloat
```

All Functions

Imports and Functions

The experiment was done using Python. I used `numpy` for numerical calculations, `matplotlib` for graphs, `scipy` for statistical tests and ODR, and `uncertainties` for propagating measured uncertainties.

Uncertainties are as given below:

```
sigma_N_turns = 1          # miscounting a partial turn, etc.
sigma_R_cm = 0.25          # ruler read in 0.5 cm increments, so half that
sigma_h_cm = 0.25          # ruler read in 0.5 cm increments, so half that
sigma_emf_frac = 0.02      # 2% of oscilloscope voltage reading
```

```
mu_0 = constants.mu_0

calib_z_cm = np.array([14, 13, 12, 11, 10, 9.5, 9, 8.5, 8, 7.5, 7, 6.5, 6, 5.5,
5, 4.5, 4, 3.5, 3])
calib_B_mT = np.array([0.021, 0.049, 0.098, 0.153, 0.245, 0.318, 0.385, 0.466,
0.527,
0.671, 0.797, 1.01, 1.23, 1.65, 2.16, 2.95, 3.89,
5.16, 7.49])
```

```
sigma_z_cm = 0.2 # given uncertainty on z
sigma_B_mT = 0.005 # given uncertainty on B
```

odr_fit still here even though calibration doesn't use it anymore -kept for the best-fit-line checks later in **Results**.

```
@dataclass
class odr_fit_result:
    parameters: np.ndarray
    parameter_errors: np.ndarray
    r_squared: float
    rmse: float
    reduced_chi_squared: float
    converged: bool
    x_nom: np.ndarray
    y_nom: np.ndarray
    x_err: np.ndarray
    y_err: np.ndarray

def odr_fit(x: list[ufloat], y: list[ufloat], model_func, beta0: list[float]) -
> odr_fit_result:
    x_nom = np.array([v.n for v in x])
    x_err = np.array([v.s for v in x])
    y_nom = np.array([v.n for v in y])
    y_err = np.array([v.s for v in y])

    data = odr.RealData(x_nom, y_nom, sx=x_err, sy=y_err)
    model = odr.Model(model_func)
    odr_result = odr.ODR(data, model, beta0=beta0).run()

    parameters = odr_result.beta
    parameter_errors = odr_result.sd_beta
    converged = odr_result.info in (1, 2, 3)
    if not converged:
        print(f"did not converge (info={odr_result.info})")

    y_pred = model_func(parameters, x_nom)
    residuals = y_nom - y_pred
    ss_res = np.sum(residuals**2)
    ss_tot = np.sum((y_nom - np.mean(y_nom)) ** 2)
    r_squared = 1 - ss_res / ss_tot if ss_tot != 0 else np.nan
    rmse = np.sqrt(np.mean(residuals**2))
    reduced_chi_squared = odr_result.res_var
```

```

return odr_fit_result(
    parameters=parameters,
    parameter_errors=parameter_errors,
    r_squared=r_squared,
    rmse=rmse,
    reduced_chi_squared=reduced_chi_squared,
    converged=converged,
    x_nom=x_nom,
    y_nom=y_nom,
    x_err=x_err,
    y_err=y_err,
)

```

```

def chi_squared_test(chi2_stat, dof):
    p_value = stats.chi2.sf(chi2_stat, dof)
    print(f"chi-squared: {chi2_stat:.3f} (dof = {dof})")
    print(f"reduced chi-squared: {chi2_stat/dof:.3f}")
    print(f"p-value: {p_value:.4f}")
    if p_value > 0.05:
        print("results AGREE with the model (p > 0.05)")
    else:
        print("results DISAGREE with the model (p <= 0.05)")
    return p_value

def sigma_test(k1, sk1, k2, sk2, label="parameter"):
    sigma_diff = abs(k1 - k2) / np.sqrt(sk1**2 + sk2**2)

    print(f"\nSIGMA TEST ({label})")
    print(f"measured value      : {k1:.3f} +/- {sk1:.3f}")
    print(f"theoretical value     : {k2:.3f} +/- {sk2:.3f}")
    print(f"discrepancy           : {sigma_diff:.2f} sigma")

    if sigma_diff <= 3.0:
        print("results AGREE within experimental uncertainty")
    else:
        print("results DISAGREE")

    return sigma_diff

```

theory prediction + comparison functions. A p-value greater than 0.05 means data is statistically consistent with the theoretical model. weighted mean ratio close to one indicates that the overall experimental scale is consistent with theory.

```
PEAK_EMF_COEFF = (3 / 4) * (4 / 5) ** 2.5 # ~0.4293

def theory_emf_peak(N, v, R):
    return PEAK_EMF_COEFF * N * v * mu_0 * m_dipole / R ** 2

def compare_to_theory(x_vals, y_measured_v, y_theory, title):
    x_nom = np.array([v.n for v in x_vals])
    x_err = np.array([v.s for v in x_vals])
    sigma_y = y_measured_v * sigma_emf_frac
    y_nom = y_measured_v

    residuals = y_nom - y_theory

    chi2_stat = np.sum((residuals / sigma_y) ** 2)
    dof = len(x_nom)
    print(f"\n~~~{title.upper()}~~~")
    chi_squared_test(chi2_stat, dof)

    ratio = y_nom / y_theory
    ratio_err = ratio * (sigma_y / y_nom)
    weights = 1 / ratio_err ** 2
    mean_ratio = np.sum(ratio * weights) / np.sum(weights)
    mean_ratio_err = 1 / np.sqrt(np.sum(weights))
    sigma_test(mean_ratio, mean_ratio_err, 1.0, 0.0, label=f"{title}: data/
theory ratio")

    return x_nom, x_err, y_nom, sigma_y
```

best-fit-line diagnostic for comparison against the fixed theory curve above.

```
def linear_model(p, x):
    return p[0] * x + p[1]

def power_model(p, x):
    return p[0] * x ** p[1]
```

```

def plot_with_bestfit(x_nom, x_err, y_nom, sigma_y, y_theory, best_fit_result,
                    model_func, xlabel, title, filename_prefix):
    order = np.argsort(x_nom)
    x_smooth = np.linspace(min(x_nom), max(x_nom), 200)
    y_bestfit_smooth = model_func(best_fit_result.parameters, x_smooth)

    plt.figure(figsize=(7, 5))
    plt.errorbar(x_nom, y_nom, xerr=x_err, yerr=sigma_y, fmt="o", capsize=3,
                label="Experimental Data (Oscilloscope)")
    plt.plot(x_nom[order], y_theory[order], "k--o", markersize=4,
            label="Theory (no free parameters)")
    plt.plot(x_smooth, y_bestfit_smooth, "g-",
            label=f"Best fit ($R^2$={best_fit_result.r_squared:.3f})")
    plt.xlabel(xlabel)
    plt.ylabel("Peak emf $\epsilon$ (V)")
    plt.title(title)
    plt.grid(True, alpha=0.6)
    plt.legend()
    plt.tight_layout()
    plt.savefig(f"{filename_prefix}_fit.png", dpi=300)
    plt.show()

```

Aim

To determine how the peak emf induced by a magnet falling through a coil depends on the number of turns N , the coil radius R and the magnet's speed s , and to compare against prediction from a calibrated dipole moment.

Theory

Faraday's law:

$$\epsilon = -\frac{d\Phi_B}{dt}, \quad \Phi_B = \int B_{\perp} dA$$

Getting the dipole moment: The magnet is treated as a point magnetic dipole with magnetic moment (m). The magnetic field along the dipole axis is therefore

$$B(z) = \frac{\mu_0 m}{2\pi z^3}$$

Rearranging gives

$$k = Bz^3 = \frac{\mu_0 m}{2\pi}$$

If the point-dipole approximation is valid, (k) should remain approximately constant across the calibration data. Rather than performing a fit, the value of (k) was calculated for each of the 19 calibration points and averaged. The dipole moment was then determined from

$$m = \frac{2\pi(k)}{\mu_0}$$

Deriving the peak emf. For a coaxial circular loop of radius (R) , the magnetic flux produced by a dipole at an axial distance (z) is

$$\Phi(z) = \frac{\mu_0 m R^2}{2(R^2 + z^2)^{3/2}}$$

Applying Faraday's law and using the chain rule, with $(v=dz/dt)$,

$$\varepsilon = -N \frac{d\Phi}{dt} = -Nv \frac{d\Phi}{dz} = \frac{3}{2} \frac{Nv\mu_0 m R^2 z}{(R^2 + z^2)^{5/2}}$$

To determine the position at which the emf reaches its maximum, the expression

$$\frac{z}{(R^2 + z^2)^{5/2}}$$

is maximised. Setting its derivative to zero gives

$$R^2 - 4z^2 = 0$$

so the peak occurs at

$$z = \frac{R}{2}$$

Substituting this into the expression for the emf gives

$$\frac{3}{4} \left(\frac{4}{5} \right)^{5/2} \frac{Nv\mu_0 m}{R^2}$$

or

$$\frac{3}{4} \left(\frac{4}{5} \right)^{5/2} \frac{Nv\mu_0 m}{R^2} \approx 0.4293 \frac{Nv\mu_0 m}{R^2}$$

Procedure

We wound a coil around one end of the Perspex tube, sanded enamel off both leads, hooked to scope, mounted tube vertically. We then dropped the magnet, adjusted scope scale/trigger till we got a clean two-lobed trace. The larger peak corresponded to the magnet exiting the coil, as the magnet was accelerating throughout its fall (gravity doesn't take breaks).

Peak emf read as $\frac{V_{pk-pk}}{2}$ off the scope, per demonstrator's instruction. oscilloscope reading uncertainty taken as 2% of the reading.

Dependence on number of turns (N)

The coil radius and drop height were kept constant while the number of turns was varied across nine values from 10 to 50 turns. The peak emf was recorded for each turn count.

Dependence on coil radius (R)

The magnet was dropped with the north pole facing upwards. Coils were wound onto sleeves of increasing diameter.

Because the coil position changed slightly between sleeves, the drop height was measured individually for each coil rather than assuming a single fixed height. The magnet speed at the coil was then calculated from

$$s = \sqrt{2gh}$$

Dependence on magnet speed (s)

The number of turns and coil radius were kept constant while the drop height was varied in approximately 5 cm increments, from 4.9 cm to 44.9 cm. The magnet speed at the coil was calculated using

$$s = \sqrt{2gh}$$

The uncertainties of measured quants are as follows (R and h were read off a ruler marked in 0.5 cm steps, so we took half that as the reading uncertainty):

Quantity	Uncertainty
Number of turns (N)	(1) turn
Coil radius (R)	(0.25) cm
Drop height (h)	(0.25) cm
Calibration distance (z)	(0.2) cm
Calibration field (B)	(0.005) mT
Oscilloscope emf	(2%) of reading

For the oscilloscope, the uncertainty used in the theory comparisons was

$$\sigma_{\varepsilon} = 0.02\varepsilon.$$

For the speed experiment,

$$v = \sqrt{2gh}$$

so

$$\sigma_v = \left| \frac{dv}{dh} \right| \sigma_h = \frac{g}{v} \sigma_h.$$

This was implemented using `ufloat`.

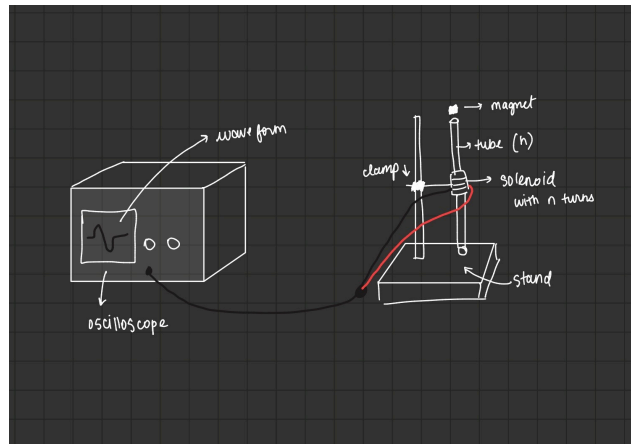


Figure 1: Sketch

Results

Dipole moment calibration

```
z_m = calib_z_cm / 100
B_T = calib_B_mT / 1000

k_values = B_T * z_m ** 3          # should be ~constant if B = k/z^3 holds
k_mean = k_values.mean()
k_err = k_values.std(ddof=1) / np.sqrt(len(k_values)) # SE of the mean

m_dipole = k_mean * 2 * np.pi / mu_0
sigma_m = k_err * 2 * np.pi / mu_0

print(f"~~~DIPOLE MOMENT CALIBRATION~~~")
print(f"m = {m_dipole:.4f} +/- {sigma_m:.4f} A m^2")
print(f"spread: {k_values.std(ddof=1)/k_mean*100:.1f}% of the mean")
```

```
~~~DIPOLE MOMENT CALIBRATION~~~
m = 1.1784 +/- 0.0727 A m^2
spread: 26.9% of the mean
```

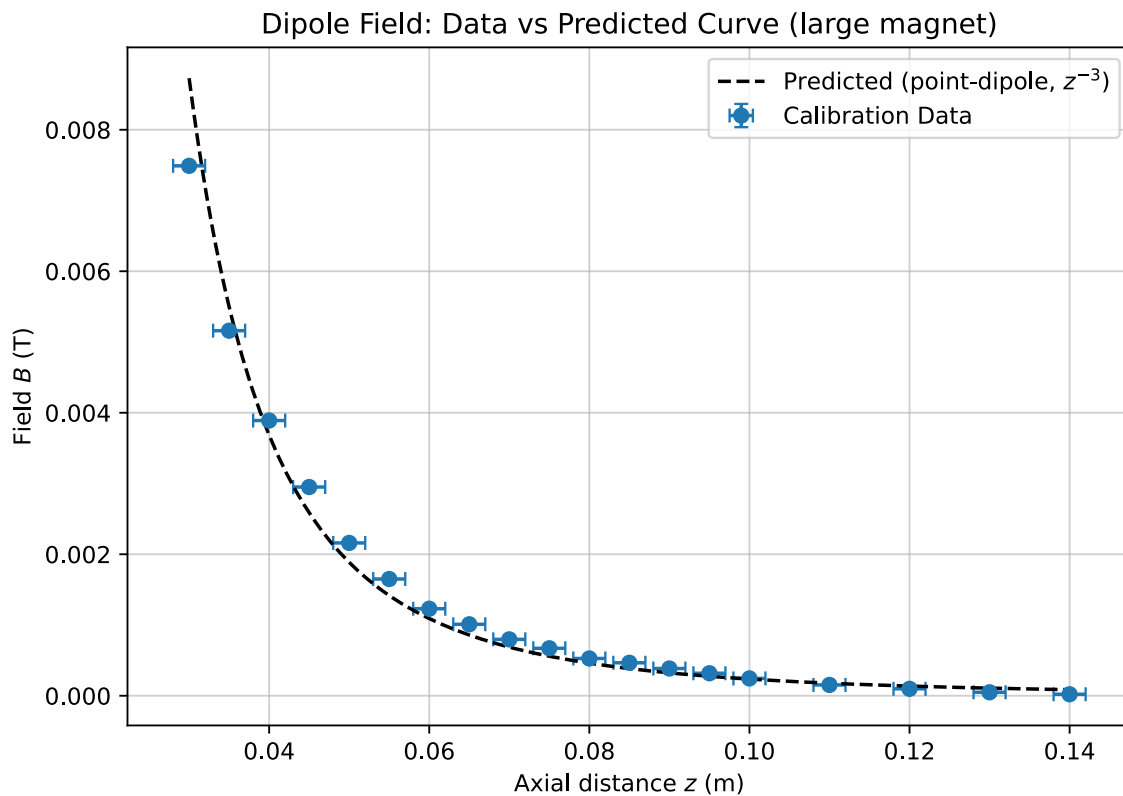
```
z_smooth = np.linspace(min(z_m), max(z_m), 200)
B_theory = (mu_0 / (2 * np.pi)) * m_dipole / z_smooth ** 3
```



```

plt.figure(figsize=(7, 5))
plt.errorbar(z_m, B_T, xerr=sigma_z_cm / 100, yerr=sigma_B_mT / 1000,
            fmt="o", capsize=3, label="Calibration Data")
plt.plot(z_smooth, B_theory, "k--", label="Predicted (point-dipole,  $z^{-3}$ )")
plt.xlabel("Axial distance  $z$  (m)")
plt.ylabel("Field  $B$  (T)")
plt.title("Dipole Field: Data vs Predicted Curve (large magnet)")
plt.grid(True, alpha=0.6)
plt.legend()
plt.tight_layout()
plt.savefig("graphs/dipole_calibration_fit.png", dpi=300)
plt.show()

```

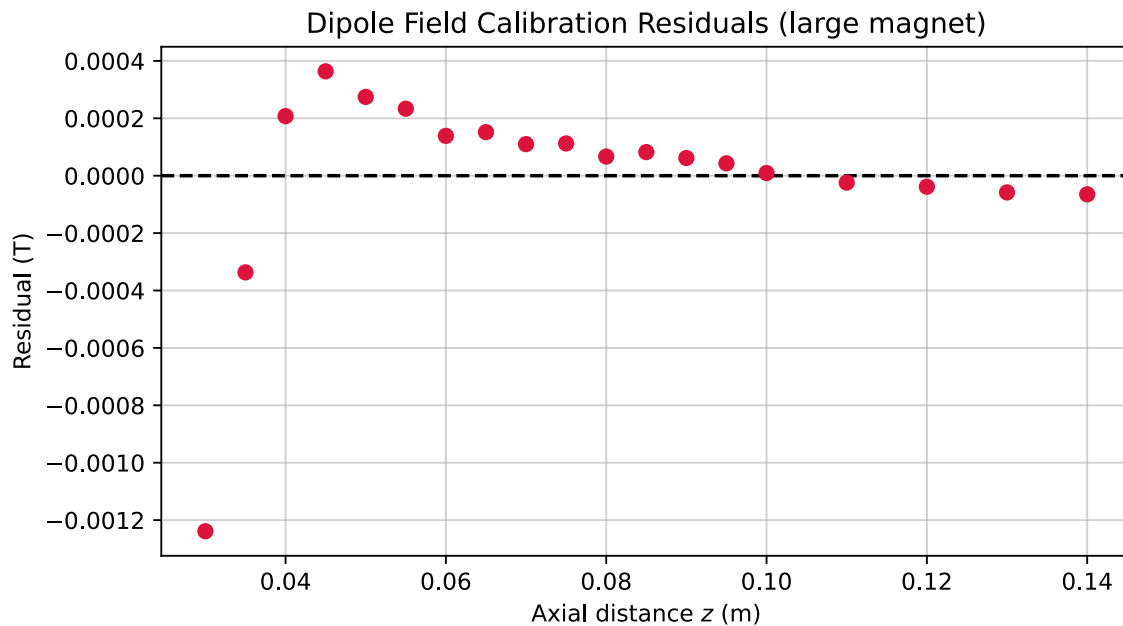


```

B_predicted_at_data = (mu_0 / (2 * np.pi)) * m_dipole / z_m ** 3
calib_residuals = B_T - B_predicted_at_data
plt.figure(figsize=(7, 4))
plt.errorbar(z_m, calib_residuals, yerr=sigma_B_mT / 1000, fmt="o", capsize=3,
            color="crimson")

```

```
plt.axhline(0, color="black", linestyle="--")
plt.xlabel("Axial distance $z$ (m)")
plt.ylabel("Residual (T)")
plt.title("Dipole Field Calibration Residuals (large magnet)")
plt.grid(True, alpha=0.6)
plt.tight_layout()
plt.savefig("graphs/dipole_calibration_residuals.png", dpi=300)
plt.show()
```



For each calibration point $k = Bz^3$ was calculated. If the magnetic field follows the ideal point-dipole relationship $B \propto z^{-3}$, these values should be approximately constant.

The mean value of (k) was calculated from all 19 calibration points, and the standard error of the mean was used as the uncertainty in the mean. This gave a calibrated magnetic dipole moment of

$$m = (1.1784 \pm 0.0727) \text{ A} \cdot \text{m}^2$$

The individual (k) -values had a spread of approximately 27% relative to their mean. The largest deviations occurred at the closest measurement points, where the measured field showed increasing deviation from the ideal z^{-3} dependence — not too surprising, since “point” dipole is a bit of a stretch up close.

emf vs number of turns N

```
R_fixed_N = ufloat((3.4)/2, sigma_R_cm) / 100 # m
h_fixed_N = ufloat(48.4, sigma_h_cm) / 100 # m
```

```

v_fixed_N = np.sqrt(2 * constants.g * h_fixed_N.n) # speed at coil, from
free fall

turns_list = [10, 15, 20, 25, 30, 35, 40, 45, 50]
turns_N = [ufloat(n, sigma_N_turns) for n in turns_list]
emf_vs_N_measured = np.array([66, 116, 128, 166, 174, 226, 252, 260, 276]) / 2 /
1000 # V

y_theory_N = np.array([theory_emf_peak(N, v_fixed_N, R_fixed_N.n) for N in
turns_list])
xN, xerrN, yN, syN = compare_to_theory(
    turns_N, emf_vs_N_measured, y_theory_N, title="Peak emf vs Number of Turns",
)

```

```

~~~PEAK EMF VS NUMBER OF TURNS~~~
chi-squared: 30664.319 (dof = 9)
reduced chi-squared: 3407.147
p-value: 0.0000
results DISAGREE with the model (p <= 0.05)

SIGMA TEST (Peak emf vs Number of Turns: data/theory ratio)
measured value      : 0.461 +/- 0.003
theoretical value   : 1.000 +/- 0.000
discrepancy         : 174.58 sigma
results DISAGREE

```

The experimental data demonstrated an approximately linear increase in peak emf as the number of turns increased from 10 to 50. Comparing the data to the absolute theoretical prediction gave a chi-squared statistic of $\chi^2 = 30664.319$ (dof = 9, reduced $\chi^2 = 3407.147$, $p = 0.0000$). A sigma test on the data-to-theory ratio gave a measured value of 0.461 ± 0.003 (discrepancy of 174.58σ).

emf vs coil radius R

```

# emf vs coil radius R
# north side up
N_fixed_R = 10
h = np.mean([48.3, 49.4, 49.2, 49.1, 49.2]) # cm
h_R = ufloat(h / 100, sigma_h_cm / 100)
d_cm = [2.5, 3.4, 4.8, 6.0, 8.9] #cm diameter
radii_cm = [d / 2 for d in d_cm] # cm

radii_R = [ufloat(r / 100, sigma_R_cm / 100) for r in radii_cm] # m

```

```

speed_R = np.sqrt(2 * constants.g * h_R.n)

emf_vs_R_measured = np.array([172, 124, 68, 44.8, 26.4]) / 2 / 1000 # pk-pk mV
-> single-peak V

y_theory_R = np.array([
    theory_emf_peak(N_fixed_R, speed_R, R.n)
    for R in radii_R
])

xR, xerrR, yR, syR = compare_to_theory(
    radii_R,
    emf_vs_R_measured,
    y_theory_R,
    title="Peak emf vs Coil Radius",
)

```

```

~~~PEAK EMF VS COIL RADIUS~~~
chi-squared: 723.329 (dof = 5)
reduced chi-squared: 144.666
p-value: 0.0000
results DISAGREE with the model (p <= 0.05)

SIGMA TEST (Peak emf vs Coil Radius: data/theory ratio)
measured value      : 0.899 +/- 0.008
theoretical value   : 1.000 +/- 0.000
discrepancy         : 12.27 sigma
results DISAGREE

```

The measured peak emf decreased as the coil radius increased across diameters ranging from 2.5 cm to 8.9 cm, matching the expected inverse relationship $\varepsilon_{\text{peak}} \propto R^{-2}$. Comparing against theory yielded $\chi^2 = 723.329$ (dof = 5, reduced $\chi^2 = 144.666$, $p = 0.0000$). A data-to-theory ratio sigma test gave 0.899 ± 0.008 (12.27 σ discrepancy). This discrepancy occurs because the finite size of the neodymium magnet (approximately 20 mm wide) makes the point-dipole approximation less accurate for smaller coil radii down to 1.25 cm.

emf vs magnet speed s

```

N_fixed_s = 10
R_fixed_s = ufloat((4.8) / 2), sigma_R_cm) / 100 # m

```

```

heights_cm = [44.9, 39.9, 34.9, 29.9, 24.9, 19.9, 14.9, 9.9, 4.9] # cm
heights_m = [ufloat(hh / 100, sigma_h_cm / 100) for hh in heights_cm]
speeds_s = [ufloat(np.sqrt(2 * constants.g * hh.n),
                    constants.g * hh.s / np.sqrt(2 * constants.g * hh.n)) for
hh in heights_m]

emf_vs_s_measured = np.array([74, 68, 62, 57.2, 47.6, 44, 40.8, 34.8, 29.6]) /
2 / 1000 # mV -> V

y_theory_s = np.array([theory_emf_peak(N_fixed_s, v.n, R_fixed_s.n) for v in
speeds_s])

xs, xerrs, ys, sys_ = compare_to_theory(
    speeds_s, emf_vs_s_measured, y_theory_s, title="Peak emf vs Magnet Speed",
)

```

```

~~~PEAK EMF VS MAGNET SPEED~~~
chi-squared: 306.792 (dof = 9)
reduced chi-squared: 34.088
p-value: 0.0000
results DISAGREE with the model (p <= 0.05)

SIGMA TEST (Peak emf vs Magnet Speed: data/theory ratio)
measured value      : 1.088 +/- 0.007
theoretical value   : 1.000 +/- 0.000
discrepancy         : 12.04 sigma
results DISAGREE

```

The measured peak emf increased linearly as the drop height—and consequently the magnet speed calculated via $s = \sqrt{2gh}$ —increased from 4.9 cm to 44.9 cm. The comparison against theory produced $\chi^2 = 306.792$ (dof = 9, reduced $\chi^2 = 34.088$, $p = 0.0000$). A sigma test on the ratio gave 1.088 ± 0.007 (12.04 σ discrepancy), which is primarily a consequence of small assigned uncertainties rather than a large physical disagreement in magnitude.

Line of best fit

For the coil-radius data, we used a power-law model $\varepsilon = aR^b$. The fitted exponent came out to -1.53 ± 0.07 , against a theoretical -2 . This is the clearest shape mismatch of the three, and — as flagged above — fits neatly with the point-dipole approximation wearing thin once the coil radius gets close to the magnet’s own size.

```

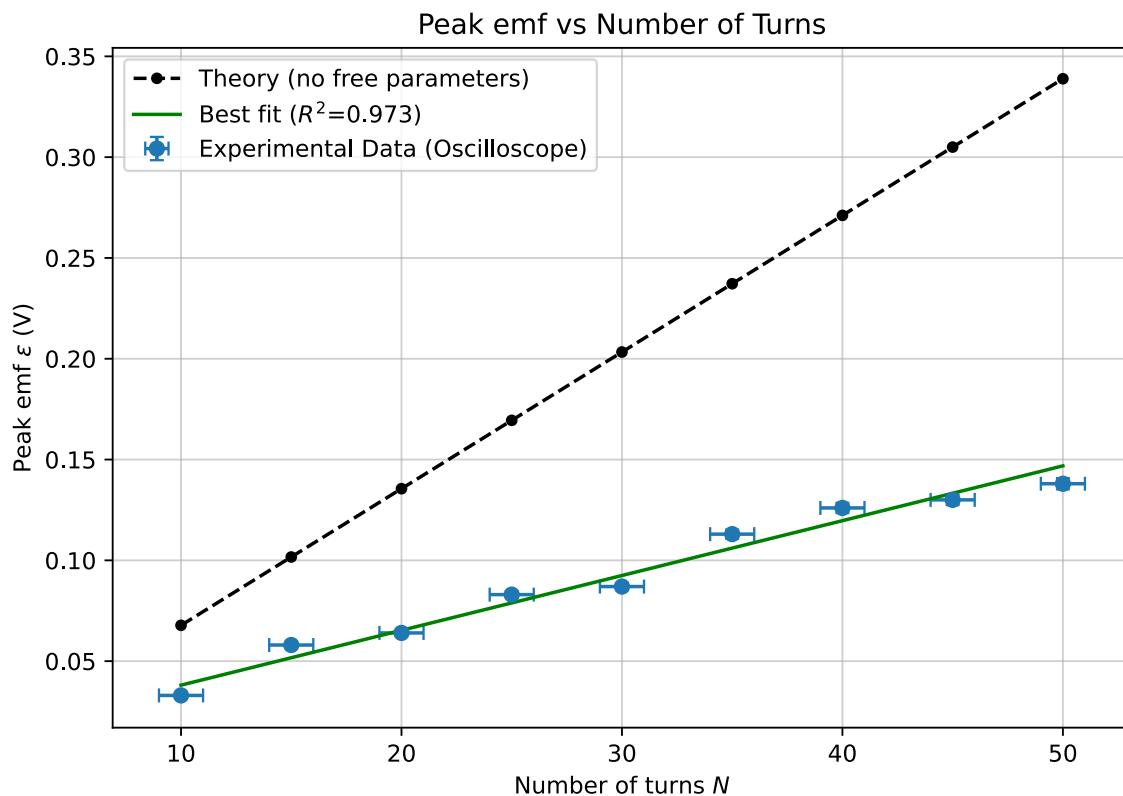
print("\n-- N: linear fit emf = a*N + b --")
res_N = odr_fit(turns_N, [ufloat(v, s) for v, s in zip(yN, syN)], linear_model,
beta0=[0.001, 0])
print(f"a      (slope)      =      {res_N.parameters[0]:.6f}      +/-
{res_N.parameter_errors[0]:.6f}")
print(f"b      (offset)      =      {res_N.parameters[1]:.6f}      +/-
{res_N.parameter_errors[1]:.6f}")
print(f"R^2 = {res_N.r_squared:.4f}")
plot_with_bestfit(xN, xerrN, yN, syN, y_theory_N, res_N, linear_model,
                  "Number of turns $N$", "Peak emf vs Number of Turns",
                  "graphs/emf_vs_N")

```

```

-- N: linear fit emf = a*N + b --
a (slope) = 0.002719 +/- 0.000163
b (offset) = 0.010901 +/- 0.004888
R^2 = 0.9727

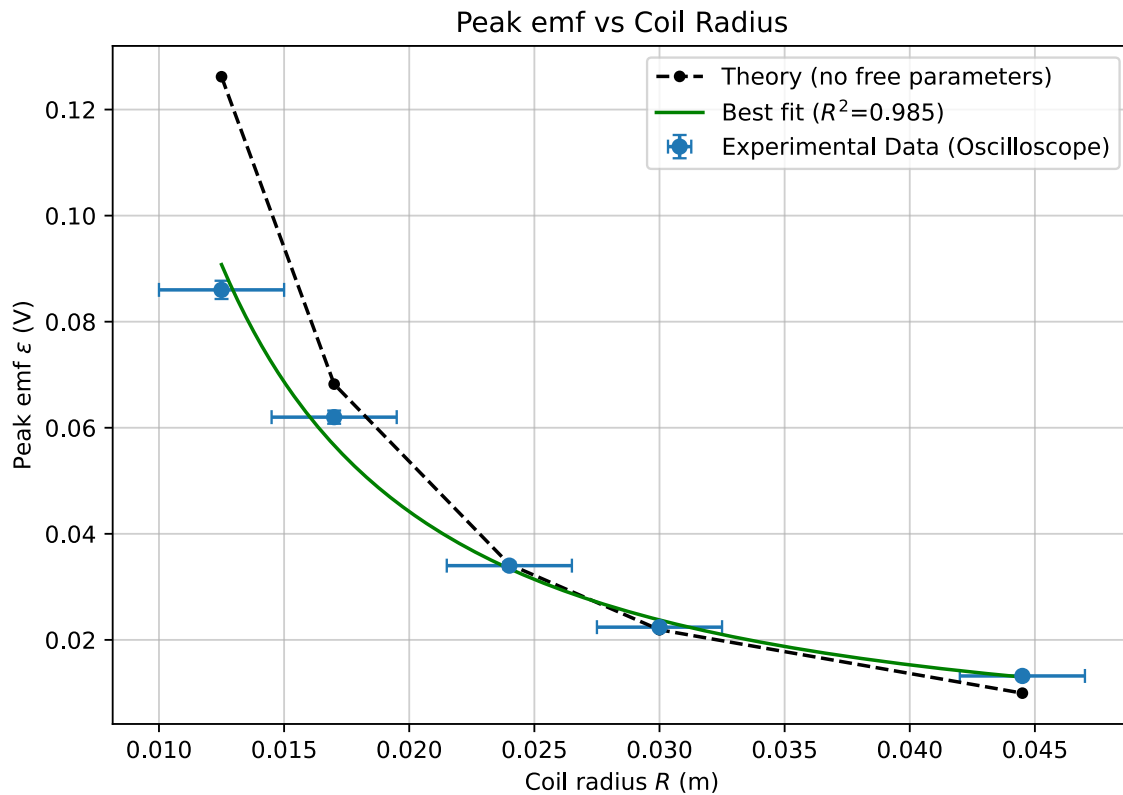
```



For the number-of-turns data, a linear model of the form $\varepsilon = aN + b$ was used. The high (R^2) value indicates that the experimental data follow a strong linear trend, consistent with the predicted dependence on (N).

```
print("\n-- R: power fit emf = a*R^b --")
res_R = odr_fit(radii_R, [ufloat(v, s) for v, s in zip(yR, syR)], power_model,
beta0=[0.0001, -2])
print(f"a      (scale)      =      {res_R.parameters[0]:.8f}      +/-
{res_R.parameter_errors[0]:.8f}")
print(f"b      (exponent)    =      {res_R.parameters[1]:.4f}      +/-
{res_R.parameter_errors[1]:.4f}")
print(f"R^2 = {res_R.r_squared:.4f}")
plot_with_bestfit(xR, xerrR, yR, syR, y_theory_R, res_R, power_model,
                  "Coil radius $R$ (m)", "Peak emf vs Coil Radius",
"graphs/emf_vs_R")
```

```
-- R: power fit emf = a*R^b --
a (scale) = 0.00011061 +/- 0.00002505
b (exponent) = -1.5313 +/- 0.0658
R^2 = 0.9852
```

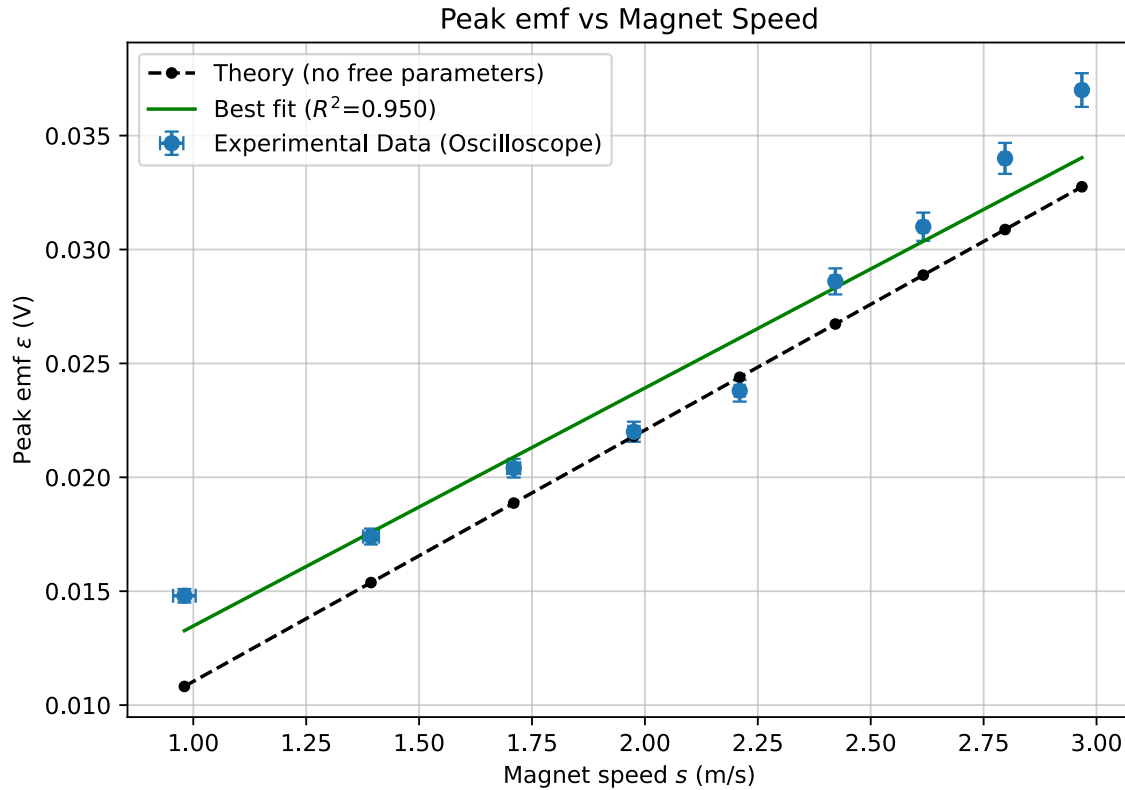


For the speed data, a linear model of the form $\varepsilon = as + b$ was used. The high (R^2) value demonstrates a strong linear relationship between peak emf and magnet speed, in agreement with the theoretical prediction.

```
print("\n-- s: linear fit emf = a*v + b --")
res_s = odr_fit(speeds_s, [ufloat(v, s) for v, s in zip(ys, sys_)], linear_model,
beta0=[0.05, 0])
print(f"a          (slope)          =          {res_s.parameters[0]:.6f}          +/-
{res_s.parameter_errors[0]:.6f}")
print(f"b          (offset)          =          {res_s.parameters[1]:.6f}          +/-
{res_s.parameter_errors[1]:.6f}")
print(f"R^2 = {res_s.r_squared:.4f}")
plot_with_bestfit(xs, xerrs, ys, sys_, y_theory_s, res_s, linear_model,
                  "Magnet speed $$ (m/s)", "Peak emf vs Magnet Speed",
"graphs/emf_vs_s")
```

```
-- s: linear fit emf = a*v + b --
a (slope) = 0.010448 +/- 0.000918
```


b (offset) = 0.003026 +/- 0.001801
 $R^2 = 0.9504$



All three datasets had (R^2) values above 0.94 indicating that the measurements were internally consistent and exhibited relatively little scatter. The (N) and (s) datasets showed strong agreement with their predicted functional forms, while the radius data showed the main deviation in shape.

Overall Observations

The three experiments show the expected qualitative relationships:

Variable changed	Expected dependence	Experimental trend
N	$\epsilon_{\max} \propto N$	emf increased
R	$\epsilon_{\max} \propto R^{-2}$	emf decreased strongly
s	$\epsilon_{\max} \propto s$	emf increased

So,

$$\epsilon_{\max} \propto \frac{Ns}{R^2}$$

Waveform and Polarity

When the magnet falls through the coil, the emf changes sign as the magnet approaches and then moves away from the centre of the coil. This produces a two-lobed waveform on the oscilloscope.

Before the magnet reaches the centre, the magnetic flux through the coil is increasing. After the magnet passes the centre, the flux is decreasing, so the sign of $\frac{d\Phi_B}{dt}$ changes.

Reversing the magnet polarity reverses the direction of the magnetic field and therefore reverses the polarity of the induced emf. The magnitude should remain approximately the same if the magnet follows the same path.

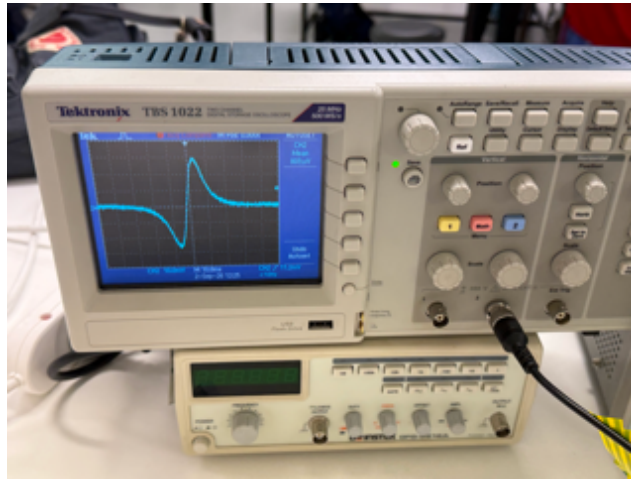


Figure 2: reversed polarity

The waveform does not have to be perfectly symmetric. The magnet is accelerating as it falls, so its speed is different on either side of the coil.

Limitations & Possible Sources of Discrepancy

There are several reasons why the experimental results do not follow the theoretical predictions perfectly.

1. The magnet is not a perfect point dipole

The calibration assumes

$$B \propto z^{-3}$$

This is the point-dipole approximation. The real neodymium magnet has a finite size, so the approximation becomes less accurate when the magnet is close to the coil or magnetic-field sensor.

This can affect the calibrated value of m in turn affecting every theoretical emf prediction.

2. The magnet may not fall perfectly along the coil axis

The theoretical model assumes that the magnet travels directly along the symmetry axis of the coil. In the real experiment, the magnet can wobble or touch the inside of the tube.

A small sideways displacement changes the magnetic flux through the coil and therefore changes the measured emf.

3. The speed was calculated rather than directly measured

The speed was obtained from

$$s = \sqrt{2gh}$$

This assumes ideal free fall. Any interaction between the magnet and the tube, air resistance, or release effects can make the actual speed different from this value.

4. Coil geometry is not perfectly ideal

The theory treats the coil as a circular loop with a well-defined radius. In practice, the wire has finite thickness and the turns are not necessarily located at exactly the same radius.

5. Peak-to-peak readings

The oscilloscope readings were recorded as peak-to-peak voltages and divided by two to obtain the single-peak magnitude rather than using the cursor to highlight exact peaks on a low resolution wave.

6. Position and drop-height consistency

The radius experiment required the coil geometry to be changed, which also changed the coil position slightly.

Conclusion

The experiment showed that increasing the number of turns increased the peak emf, increasing the magnet speed increased the peak emf, and increasing the coil radius decreased the peak emf. These trends agree with the predicted functional dependence

$$\boxed{\varepsilon_{\max} \propto \frac{Ns}{R^2}}$$

The agreement was not perfect mainly because the theoretical model assumes an ideal point dipole, ideal free fall, perfectly centred motion and an ideal circular coil, none of which we can manage to build in a lab as of now.

ANU Physics Academic Integrity Declaration Form

Name: chhaya ramnani Student ID: u8336188

Date: 3 sep 2026 Assessment Item: logbook report week 5 and 6

Collaboration with others

You are encouraged to discuss problems with other students, however you must submit your own work. The minimum penalty for plagiarism is a fail on the assessment task, and all incidents of plagiarism are recorded on a student's academic record.

If you have collaborated with others, please note these people here:

candice ng, chinatsu komada

Use of AI Tools

You may not present material produced by generative AI as your own work, as this is an academic integrity breach. If you have used AI in this assessment, the use must be declared here:

☐ No AI tools were used for this assignment

☒ AI tools were used as described below:

AI Tools Used: gemini

How AI was used (check all that apply):

☐ Mathematical calculations ☐ Problem solving ☒ Code generation
☐ Writing assistance ☐ Research ☐ Other: _____

Chat Evidence Required - you must include your complete AI conversation(s):

☒ Full chat transcript pasted below
☐ Full chat screenshots/files attached
☐ Shared link: <https://share.gemini.google/w4Amdd0qyHK5>

I declare that:

- ☒ This AI use declaration is complete and accurate
- ☒ I have included unedited records of all AI interactions
- ☒ I understand the physics concepts and can explain my work
- ☒ I remain responsible for all content in my submission

Signature:  Date: 3 sep 2026